

System Prompt Implícito

Directrices de Código para Scripts Generados por IA

v3.0 — 2025

Este documento actúa como **system prompt implícito**: pegarlo al inicio de cualquier conversación con una IA condiciona toda la generación de código sin necesidad de repetir instrucciones. Cubre entorno, prioridades, reintentos, modularidad, mapeos, datos sensibles, rutas, versionado, logs y checklist de validación.

Índice

1	Contexto de Entorno de Ejecución
2	Prioridades de Diseño
3	Política de Reintentos y Backoff
4	Estructura Obligatoria del Script
5	Plantilla Markdown — Encabezado
6	Criterio de Modularidad
7	Convenciones de Robustez
8	Convenciones de Escalabilidad
9	Sin Dependencias — Política y Excepciones
10	Política de Datos Sensibles
11	Convención de Rutas y Outputs
12	Versionado Mínimo del Script
13	Formato de Logs y Output
14	Checklist de Validación

1. Contexto de Entorno de Ejecución

Dos entornos de ejecución coexisten en cada proyecto. El agente los distingue por el valor de **MODO** y adapta el comportamiento sin generar dos versiones del script.

Parámetro	Producción	Auditoría
Plataforma	GitHub Actions	Google Colab
Quién ejecuta	cron.yml — automático	Usuario — manual
MODO	'produccion'	'auditoria'
Celdas Colab	No aplica	Una sola celda — prioridad máxima
Telegram	Activo — siempre notifica	Desactivado
Log	Consola + fichero	Solo consola
Rutas	Relativas al repo (./)	/content/drive/MyDrive/[proyecto]/
Python	3.10+	3.10+
Encoding	UTF-8	UTF-8
Timezone	UTC	UTC

Estructura estándar de repositorio:

Ruta	Contenido
<code>.github/workflows/cron.yml</code>	Schedule de ejecución automática
<code>mapeos/*.json</code>	Datos de referencia y configuración versionados
<code>scripts/*.py</code>	Scripts principales
<code>output/raw/</code>	Datos sin procesar — nombre fijo, sobrescrito en cada ejecución
<code>output/clean/</code>	Datos transformados — nombre fijo
<code>output/logs/</code>	Logs con timestamp — red de seguridad final
<code>docs/manual_operativa.html</code>	Manual de operativa del proyecto — mantenido en GitHub
<code>README.md</code>	Descripción general del proyecto

El agente asume esta estructura en todos los proyectos sin que se le indique explícitamente. El punto de encuentro entre producción y auditoría es la URL raw de /mapeos/ en GitHub.

2. Prioridades de Diseño

Prioridad	Principio	Aplicación práctica
■ 1	Robustez	Errores explícitos y con contexto. Validaciones previas. Reintentos con backoff. Sin except desnudos. Logs con nivel.
■ 2	Escalabilidad	Modular, parametrizable, sin hardcoding. Excepción: MODO y configuración Telegram se cargan desde /mapeos/telegram_config.json. Una función = una responsabilidad. Type hints.
■ 3	Sin dependencias	Diseñar para Colab primero — una sola celda. stdlib pura siempre que sea posible. Toda dependencia externa justificada explícitamente. Los mapeos JSON no son dependencias. Detalle en sección 9.

3. Política de Reintentos y Backoff

Todo script que hace I/O implementa backoff exponencial con jitter. La función base es stdlib pura. El agente distingue dos casos:

Caso 1 — Una sola fuente:

Parámetros globales al inicio del script. Una única función de reintento con esos valores. Simple, consistente, suficiente.

Caso 2 — Múltiples fuentes:

Un diccionario PERFILES_FUENTE centraliza la configuración por fuente. La función base es única — los parámetros varían. Sin duplicar código.

Fuente	max_reintentos	timeout (seg)	pausa (seg)
b1s	3	10	1.0
sec	3	15	0.5
telegram	5	5	0.0
github	3	10	0.0

Errores recuperables vs no recuperables:

Tipo de error	Recuperable	Acción
Red, timeout, OSError	■ Sí	Reintentar con backoff
HTTP 429 (rate limit)	■ Sí	Reintentar con pausa larga (60s mínimo)
HTTP 500-503 (servidor)	■ Sí	Reintentar con backoff

HTTP 400 (bad request)	■ No	Abortar — problema en el input
HTTP 401/403 (auth)	■ No	Abortar — credenciales incorrectas
ValueError / TypeError	■ No	Abortar — error de programación
KeyboardInterrupt	■ No	Dejar propagar siempre

Sin noticias es noticia: en producción el script notifica por Telegram siempre — tanto si hay éxito como si hay error. El silencio no es una opción válida. En auditoría esta notificación se desactiva con MODO = 'auditoria'.

■ Los valores de *PERFILES_FUENTE* son base inicial. Se refinan con experiencia real de cada fuente.

4. Estructura Obligatoria del Script

Todo script sigue este orden de secciones. El orden no es opcional — facilita la lectura y la generación consistente por parte del agente.

Sección	Contenido
# ENTORNO	Declaración de entorno — ver apartado 1
# VERSIÓN	Versión, fecha, modelo IA generador, registro de cambios
# MODO	MODO = 'produccion' 'auditoria' — controla Telegram, rutas y logs
# DEPENDENCIAS	Solo si hay excepciones justificadas con pip install y motivo
# CONFIGURACIÓN	Todas las variables parametrizables. Nunca dentro de funciones.
# REINTENTOS	PERFILES_FUENTE y parámetros de backoff
# MAPEOS	BASE_RAW, URLs de mapeos. Descarga con con_reintento() — fallo es fatal.
# RUTAS	Directorios adaptados por MODO. LOG_DIR siempre primero.
# VALIDACIONES	Tipos, rangos, existencia de rutas, credenciales
# FUNCIONES CORE	Lógica principal. Una función = una responsabilidad. Docstrings + type hints.
# FUNCIONES NOTIFICACIÓN	Función Telegram reutilizable. Condicionada por MODO.
# EJECUCIÓN	Orquesta todo. Incluye notificación según MODO. Bloque if __name__.

5. Plantilla Markdown — Encabezado del Script

Todo script comienza con este encabezado. En Colab/Jupyter usar celda de texto Markdown. En .py usar comentario multilínea. El agente lo genera siempre completo.

Campo	Contenido
Nombre	[NOMBRE DEL SCRIPT] — una línea: qué hace y qué problema resuelve
Entorno	Repo GitHub, MODO, Python 3.10+, OS Ubuntu 22.04
Versión	v1.0 — YYYY-MM-DD — Generado con [modelo IA]
Objetivo	Qué hace / por qué existe / input → output
MODO	produccion → Telegram activo / auditoria → Telegram desactivado
Dependencias	Sin dependencias externas — o EXCEPCIÓN: pip install X — MOTIVO: ...

Configuración	Tabla: Variable / Tipo / Default / Descripción
Mapeos	Tabla: Archivo / URL raw GitHub / Contenido
Rutas	Tabla: Ruta / Uso
Salida esperada	[INFO] / [OK] — descripción del output normal
Limitaciones	Lista de limitaciones conocidas

6. Criterio de Modularidad

La modularidad responde a un problema concreto — no es un objetivo estético. El agente aplica el nivel mínimo necesario según estos tres umbrales:

Nivel	Cuándo	Estructura	Descripción
1	Script único, < 150 líneas, no comparte nada	<code>script.py</code>	Un archivo. Secciones separadas por comentarios.
2	> 150 líneas o 2+ responsabilidades distintas	<code>main.py</code> + <code>utils.py</code>	<code>main.py</code> orquesta. <code>utils.py</code> contiene funciones reutilizables sin estado.
3	Varios scripts del mismo repo comparten funciones	<code>main.py</code> + <code>utils/</code>	Código compartido en <code>utils/</code> . Nunca duplicado entre scripts.

Estructura `utils/` en nivel 3:

Módulo	Contenido
<code>utils/http.py</code>	<code>con_reintento()</code> y llamadas de red
<code>utils/telegram.py</code>	Notificación Telegram
<code>utils/parsers.py</code>	Transformación y limpieza de datos
<code>utils/storage.py</code>	Lectura y escritura de ficheros

Regla de oro: si dos scripts copian la misma función, es el momento de crear `utils/`. Antes no. No modularizar scripts de un solo uso, prototipos o scripts que ya funcionan y no se reutilizarán.

7. Convenciones de Robustez

Nivel	Tipo	Comportamiento
1	Validación de inputs	Antes de ejecutar nada. Tipos, rangos, existencia de rutas.
2	Errores de red / IO	<code>con_reintento()</code> siempre. Descarga de mapeos es crítica — fallo aborta antes de ejecutar.
3	Errores de datos	Después de recibir respuesta. Respuesta vacía, mal formada o inesperada.
4	Errores inesperados	En el bloque <code>if __name__</code> . Loguear con [FATAL] y relanzar siempre.

Jerarquía de criticidad:

Componente	Fallo fatal	Comportamiento
------------	-------------	----------------

Descarga de mapeos	■ Sí	Abortar antes de ejecutar
Llamada a API principal	■ Sí	Reintentar, luego abortar
Escritura de output	■ Sí	Reintentar, luego abortar
Notificación Telegram	■■■ Parcial	Reintentar con <code>con_reintento()</code> . Si falla → registrar en log. El log es la red de seguridad

El log siempre se escribe independientemente de lo que ocurra. Es la red de seguridad final. Nunca usar `except` desnudos ni mensajes sin contexto.

8. Convenciones de Escalabilidad

Elemento	Directiva
MODO	Controla el comportamiento del script sin duplicar código. Siempre al inicio.
BASE_RAW	URL base del repositorio GitHub. El agente construye todas las URLs de mapeos a partir de ella.
MAPEOS	Diccionario centralizado con todas las URLs raw. Nunca construidas dentro de funciones.
PERFILES_FUENTE	Única fuente de verdad para parámetros de reintento por fuente. Extensible sin tocar el código base.
Funciones	Toda función tiene docstring y type hints. Una función = una responsabilidad.
Hardcoding	Prohibido. Excepción documentada: telegram_config.json en /mapeos/.

9. Sin Dependencias — Política y Excepciones

Nivel	Tipo	Instalación	Funciona en
1	stdlib pura	Ninguna	Todos los entornos
2	Dependencia externa justificada	!pip install — versión fijada — misma celda	Colab + cron.yml
3	Mapeos JSON	Ninguna — URL raw GitHub	Todos los entornos

El agente detecta el entorno y adapta el formato de instalación: !pip install en Colab / pip install en cron.yml de GitHub Actions. Los mapeos JSON nunca son dependencias — siempre URL raw GitHub. Toda dependencia externa tiene versión fijada y justificación escrita.

10. Política de Datos Sensibles

Todo dato sensible tiene un lugar definido. El agente consulta esta tabla antes de generar cualquier bloque de configuración. Nunca hardcodea credenciales.

Dato	Dónde vive	Producción	Auditoría Colab
Token Telegram	/mapeos/telegram_config.json	URL raw GitHub	URL raw GitHub
Chat ID Telegram	/mapeos/telegram_config.json	URL raw GitHub	URL raw GitHub
API Key BLS	Variable de entorno	GitHub Secrets	input()
Cualquier secreto crítico	Nunca en repo público	GitHub Secrets	input()

telegram_config.json contiene únicamente bot_token y chat_id. Nunca contiene otros secretos críticos. El repositorio es público — decisión consciente asumiendo que el riesgo es aceptable.

11. Convención de Rutas y Outputs

Directorio	Contenido	Nombre fichero
output/raw/	Datos sin procesar — respuesta original de la API	[proyecto]_raw.json
output/clean/	Datos transformados, listos para análisis	[proyecto]_clean.csv
output/logs/	Log de ejecución con timestamp UTC en cada línea	[proyecto]_log.txt

Nombre fijo sin timestamp — el fichero se sobrescribe en cada ejecución. El versionado lo gestiona Git mediante commits automáticos, no el nombre del fichero. LOG_DIR se crea siempre primero. Rutas relativas en producción (./) — absolutas con Drive en auditoría. El cron.yml hace commit automático de output/ tras cada ejecución.

12. Versionado Mínimo del Script

Sistema	Qué versiona	Dónde
Bloque de versión en el script	El código — lógica, funciones, estructura	Encabezado del script
Commits automáticos de Git	El output — resultados de cada ejecución	GitHub — output/

Campo	Contenido
Script	[nombre del script]
Versión	1.0, 1.1, 1.2...
Fecha	YYYY-MM-DD
Generado	Claude Sonnet 3.7 / GPT-4o / etc.
Revisado	[nombre] o 'no revisado'
Cambios	v1.0 — fecha — descripción / v1.1 — fecha — descripción

El bloque de versionado es contexto para el próximo agente de IA que modifique el script. Siempre actualizar versión, fecha y registro de cambios al modificar un script existente.

13. Formato de Logs y Output

Formato estándar de cada línea de log:

[YYYY-MM-DD HH:MM:SS UTC] [NIVEL] mensaje

Nivel	Cuándo	Ejemplo
[INFO]	Progreso normal	[INFO] Conectando con BLS API...
[OK]	Operación completada con éxito	[OK] Registros procesados: 120
[WARN]	Inusual pero no fatal	[WARN] Reintentando (2/3) en 4.0s...
[ERROR]	Fallo recuperable	[ERROR] HTTP 429: rate limit en BLS
[FATAL]	Fallo no recuperable — aborta	[FATAL] Agotados 3 reintentos. Abortando.
[CONFIG]	Error en parámetros de entrada	[CONFIG] 'start_year' debe ser int
[DATOS]	Error en datos recibidos	[DATOS] Respuesta vacía para serie: LNS14000000

Resumen Telegram — formato estándar:

[proyecto] – DD/MM/YYYY HH:MM UTC | ■ OK o ■ ERROR | Registros: N | Fichero: [proyecto]_clean.csv

Producción: log en consola + fichero output/logs/. Auditoría: log solo en consola. Telegram recibe resumen ejecutivo — nunca el log completo. El log se escribe siempre independientemente de lo que ocurra.

14. Checklist de Validación

Antes de ejecutar o entregar un script generado por IA, verificar los 21 ítems:

✓	Ítem
■	Bloque ENTORNO declarado — plataforma, Python, OS, repo
■	Bloque VERSIÓN con fecha y modelo IA generador
■	MODO declarado — 'produccion' o 'auditoria'
■	Encabezado Markdown completo con todas las secciones
■	Todas las variables configurables al inicio, fuera de funciones
■	BASE_RAW y MAPEOS definidos como constantes
■	Mapeos descargados con con_reintento() — fallo es fatal y aborta
■	PERFILES_FUENTE definido para cada fuente usada en el script

■	Manejo de errores en los 4 niveles documentados
■	Sin except desnudos ni mensajes de error sin contexto
■	Todas las funciones con docstring y type hints
■	Sin dependencias externas o todas con justificación y versión fijada
■	telegram_config.json en /mapeos/ — nunca hardcodeado
■	Notificación Telegram condicionada por MODO
■	Telegram tiene fallback a log si falla — nunca fallo fatal
■	Rutas con pathlib.Path adaptadas por MODO
■	LOG_DIR creado siempre primero
■	Log escribe en fichero en producción
■	Nombre de ficheros de output fijo — sin timestamp
■	cron.yml incluye paso de commit automático de output/
■	Bloque if <code>__name__ == '__main__'</code> presente

■ ■ Pegar este documento al inicio de cada sesión con la IA para activar el system prompt implícito. Actualizar el apartado 1 si el entorno de ejecución cambia. Versión 3.0 — resultado de sesión de refinamiento colaborativo.